

INVARIANTES

Una expresión *invariante* es una restricción que debe ser cierta para cada instancia del *contexto* asociado en todo momento y para todo estado consistente del sistema (mientras se está ejecutando una operación del sistema no se encuentra en un estado consistente). Todas las expresiones invariantes de OCL son del tipo *Boolean*.

context TypeName

inv invName: expresión booleana que debe verificarse.

PRE Y POST CONDICIONES

Una precondition es una expresión booleana que debe ser cierta (*true*) en el momento en que la operación comienza su ejecución. Una postcondición es una expresión booleana que debe ser cierta (*true*) en el momento en que la operación finaliza su ejecución.

context TypeName :: operationName(param1: Type1, ...): Return Type

pre preName: expresión booleana que debe verificarse.

post postName: expresión booleana que debe verificarse.

self	Referencia un objeto en el cual la operación fue invocada.
result	Representa el objeto que retorna la operación, es de tipo Return Type.
objeto.propiedad@pre	Indica el valor de un atributo o asociación (propiedad) antes de la ejecución de una operación. Es sufijo del nombre de la propiedad.
objeto.ocIsNew()	Permite determinar si un objeto se creó durante la ejecución de una operación.

OPERACIONES DE CONSULTA

Las operaciones de consulta no cambian el estado del sistema. Sólo retornan un valor o un conjunto de valores.

context Typename::operationName(param1 : Type1, ...): Return Type

body: expresión de consulta, es de tipo Return Type

PRECEDENCIA

Operadores de mayor a menor precedencia:

'()', 'if-then-else-endif', '@pre', ':->', 'not -', '* /', '+ -', '< > <= >=', '= <>', 'and or xor', 'implies'.

OPERACIONES PRIMITIVAS PARA EL TIPO Boolean

OPERACIÓN	NOTACIÓN	TIPO DEL RESULTADO
or / and / xor	a or b / a and b / a xor b	Boolean
iguales / no iguales	a = b / a <> b	Boolean
implica	a implies b	Boolean
negación	not a	Boolean
if-then-else-endif	if a then expresión1 else expresión2 endif	Tipo de expresión1, expresión2

OPERACIONES PRIMITIVAS PARA EL TIPO Integer Y Real

OPERACIÓN	NOTACIÓN	TIPO DEL RESULTADO
iguales / no iguales	a = b / a <> b	Boolean
menor / mayor	a < b / a > b	Boolean
menor o igual	a <= b	Boolean
mayor o igual	a >= b	Boolean
suma	a + b	Integer o Real
resta	a - b	Integer o Real
multiplicación	a * b	Integer o Real

división	a / b	Real
módulo	a.mod(b)	Integer
división entera	a.div(b)	Integer
valor absoluto	a.abs()	Integer o Real
máximo de a y b	a.max(b)	Integer o Real
mínimo de a y b	a.min(b)	Integer o Real
redondeo	a.round()	Integer
base	a.floor()	Integer

OPERACIONES PRIMITIVAS PARA EL TIPO String

OPERACIÓN	NOTACIÓN	TIPO DEL RESULTADO
concatenación	string.concat(string)	String
longitud	string.size()	Integer
a minúscula	string.toLowerCase()	String
a mayúscula	string.toUpperCase()	String
substring	string.substring(int,int)	String
iguales	string1 = string2	Boolean
no iguales	string1 <> string2	Boolean

PROPIEDADES DEFINIDAS EN TODOS LOS OBJETOS: La biblioteca estándar de OCL define el tipo *OclAny*. *OclAny* es el supertipo de todos los tipos en un modelo, sus operaciones están definidas para todos los objetos de OCL.

EXPRESIÓN	TIPO DEL RESULTADO
objeto = (objeto2: OclAny)	Boolean
objeto <> (objeto2: OclAny)	Boolean
objeto.oclIsUndefined()	Boolean
objeto.oclIsKindOf(tipo: OclType)	Boolean
objeto.oclIsTypeOf(tipo: OclType)	Boolean
objeto.oclIsNew()	Boolean
objeto.oclAsType(tipo: OclType)	tipo
objeto.oclInState(nombre: StateName)	Boolean

OPERACIONES SEGÚN LA COLECCIÓN Set

OPERACIÓN	DESCRIPCIÓN (self es de tipo Set(T))
union (s: Set(T)): Set(T)	Unión de <i>self</i> y <i>s</i> .
union (b: Bag(T)): Bag(T)	Bolsa que contiene a todos los elementos de <i>self</i> y <i>b</i> .
= (s: Set(T)): Boolean	<i>true</i> si <i>self</i> y <i>s</i> poseen los mismos elementos.
intersection (s: Set(T)): Set(T)	Conjunto formado por los elementos que están en <i>self</i> y <i>s</i> .
intersection (b: Bag(T)): Set(T)	Conjunto formado por los elementos que están en <i>self</i> y <i>b</i> .
including (o: T): Set(T)	Conjunto formado por <i>self</i> + <i>o</i> .
excluding (o: T): Set(T)	Conjunto formado por <i>self</i> sin el elemento <i>o</i> .
symmetricDifference (s:Set(T)): Set(T)	Conjunto formado por los elementos de <i>self</i> o <i>s</i> , pero no de ambos.

count (o: T): Integer	Número de ocurrencias de <i>o</i> en <i>self</i> (0 o 1)
flatten (): Set(T2)	Si el tipo de <i>self</i> no es un tipo de <i>colección</i> , el resultado es <i>self</i> . Si no, el resultado es el conjunto que contiene todos los elementos de <i>self</i> .
asSet (): Set(T)	<i>self</i>
asOrderedSet (): OrderedSet(T)	Conjunto ordenado que contiene a los elementos de <i>self</i> (orden indefinido).
asSequence (): Sequence(T)	Secuencia que contiene todos los elementos de <i>self</i> (orden indefinido).
asBag (): Bag(T)	<i>Bag</i> que contiene todos los elementos de <i>self</i> .

OPERACIONES SEGÚN LA COLECCIÓN OrderedSet

OPERACIÓN	DESCRIPCIÓN (self es de tipo OrderedSet(T))
append (o: T): OrderedSet (T)	Conjunto ordenado que tiene todos los elementos de <i>self</i> , seguido por <i>o</i> .
prepend (o: T): OrderedSet (T)	Conjunto ordenado que tiene todos los elementos de <i>self</i> , precedido por <i>o</i> .
insertAt (i: Integer, o: T): OrderedSet (T)	<i>self</i> con <i>o</i> insertado en la posición <i>i</i> .
subOrderedSet (i: Integer, u: Integer): OrderedSet (T)	Subconjunto de <i>self</i> , contiene los elementos desde <i>i</i> hasta <i>u</i> inclusive.
at (i: Integer): T	<i>i</i> -ésimo element de <i>self</i> .
indexOf (o: T): Integer	La posición de <i>o</i> en la secuencia.
first (): T	Primer elemento en <i>self</i> .
last (): T	Último elemento en <i>self</i> .

OPERACIONES SEGÚN LA COLECCIÓN Bag

OPERACIÓN	DESCRIPCIÓN (self es de tipo Bag(T))
union (s: Set(T)): Bag(T)	<i>Bag</i> que contiene a todos los elementos de <i>self</i> y <i>s</i> .
union (b: Bag(T)): Bag(T)	<i>Bag</i> que contiene a todos los elementos de <i>self</i> y <i>b</i> .
= (b: Bag(T)): Boolean	<i>true</i> si <i>self</i> y <i>s</i> poseen los mismos elementos, el mismo número de veces.
intersection (s: Set(T)): Set(T)	Conjunto formado por los elementos que están en <i>self</i> y <i>s</i> .
intersection (b: Bag(T)): Bag(T)	Bolsa formada por los elementos que están en <i>self</i> y <i>b</i> (la cantidad mínima de cada uno de los elementos).
including (o: T): Bag(T)	Bolsa formada por <i>self</i> + <i>o</i> .
excluding (o: T): Bag(T)	Bolsa formada por <i>self</i> sin el elemento <i>o</i> .
count (o: T): Integer	Número de ocurrencias de <i>o</i> en <i>self</i> .
flatten (): Bag(T2)	Si el tipo de <i>self</i> no es un tipo de <i>colección</i> , el resultado es <i>self</i> . Si no, el resultado es la <i>Bag</i> que contiene todos los elementos de <i>self</i> .
asSet (): Set(T)	Conjunto que contiene todos los elementos de <i>self</i> , eliminando duplicados.
asOrderedSet (): OrderedSet(T)	Conjunto ordenado que contiene a los elementos de <i>self</i> , eliminando duplicados (orden indefinido).
asSequence (): Sequence(T)	Secuencia que contiene todos los elementos de <i>self</i> (orden indefinido).
asBag (): Bag(T)	<i>self</i> .

OPERACIONES SEGÚN LA COLECCIÓN Sequence

OPERACIÓN	DESCRIPCIÓN (self es de tipo Sequence(T))
count (o: T): Integer	Número de ocurrencias de <i>o</i> en <i>self</i> .

union (s: Sequence(T)): Sequence(T)	Secuencia formada por todos los elementos de <i>self</i> , seguida por todos los elementos de <i>s</i> .
= (s: Sequence(T)): Boolean	<i>true</i> si <i>self</i> y <i>s</i> poseen los mismos elementos, en el mismo orden.
flatten (): Sequence(T2)	Si el tipo de <i>self</i> no es un tipo de <i>colección</i> , el resultado es <i>self</i> . Si no, es la secuencia que contiene todos los elementos de <i>self</i> (orden parcial).
append (o: T): Sequence(T)	Secuencia que tiene todos los elementos de <i>self</i> , seguido por <i>o</i> .
prepend (o: T): Sequence(T)	Secuencia que tiene todos los elementos de <i>self</i> , precedido por <i>o</i> .
insertAt (i: Integer, o: T): Sequence(T)	<i>self</i> con <i>o</i> insertado en la posición <i>i</i> .
subSequence (i: Integer, u: Integer): Sequence(T)	Subconjunto de <i>self</i> , contiene los elementos desde <i>i</i> hasta <i>u</i> inclusive.
at (i: Integer): T	<i>i</i> -ésimo elemento de <i>self</i> .
indexOf (o: T): Integer	La posición de <i>o</i> en la secuencia.
first (): T	Primer elemento en <i>self</i> .
last (): T	Último elemento en <i>self</i> .
including (o: T): Sequence	Secuencia formada por <i>self</i> + <i>o</i> .
excluding (o: T): Sequence	Secuencia formada por <i>self</i> sin el elemento <i>o</i> .
asSet (): Set(T)	Conjunto que contiene todos los elementos de <i>self</i> , eliminando duplicados.
asOrderedSet (): OrderedSet(T)	Conjunto ordenado que contiene a los elementos de <i>self</i> , en el mismo orden, eliminando duplicados.
asSequence (): Sequence(T)	<i>self</i> .
asBag (): Bag(T)	Bolsa que contiene todos los elementos de <i>self</i> , incluso los duplicados.

OPERACIONES COMUNES A TODAS LAS COLECCIONES

OPERACIÓN	DESCRIPCIÓN
colección->count(objeto)	Cantidad de veces que ocurre <i>objeto</i> en <i>colección</i> .
colección->excludes(objeto)	<i>true</i> si <i>objeto</i> no pertenece a <i>colección</i>
colección->excludesAll(colección2)	<i>true</i> si todos los elementos de <i>colección2</i> no están presentes en <i>colección</i>
colección->includes(objeto)	<i>true</i> si <i>objeto</i> pertenece a <i>colección</i>
colección->includesAll(colección2)	<i>true</i> si todos los elementos de <i>colección2</i> están presentes en <i>colección</i>
colección->isEmpty()	<i>true</i> si <i>colección</i> no posee elementos
colección->notEmpty()	<i>true</i> si <i>colección</i> contiene uno o más elementos
colección->size()	El número de elementos en <i>colección</i>
colección->sum()	La suma de todos los elementos de <i>colección</i> . Los elementos deben ser de un tipo que admite la suma.
colección->select (v booleanExpresion-with-v)	El resultado es una <i>colección</i> que contiene todos los elementos de <i>collection</i> que satisfacen <i>booleanExpresion-with-v</i> . La variable <i>v</i> itera sobre todos los elementos de <i>collection</i> y <i>booleanExpresion-with-v</i> es evaluada para cada uno.
colección->collect(v expresion-with-v)	Permite especificar que una colección se deriva de otra colección. La variable <i>v</i> itera sobre la colección original, capturando en la nueva colección el resultado de la evaluación de <i>expresión-with-v</i> para cada <i>v</i> .
colección->forAll (v booleanExpresion-with-v)	Permite especificar una expresión que debe ser cierta para todos los objetos de una colección. Es posible emplear más de un iterador.

colección->exists (v booleanExpresion-with-v)	Permite especificar una expresión que debe ser cierta para al menos un objeto de la colección.
colección->iterate (elem: Type; acc: Type = expresiónInicial expresión-with-elem-and-acc)	Operación más general. La variable <i>elem</i> es el iterador y <i>acc</i> el acumulador, inicializado en el valor <i>expresiónInicial</i> . <i>Elem</i> itera sobre los elementos de <i>collection</i> . La expresión <i>expresión-with-elem-and-acc</i> se evalúa para cada <i>elem</i> y dicho resultado se almacena en <i>acc</i> .